
EVALUATING I/O BACKENDS FOR LLM CHECKPOINT LOADING ACROSS FORMATS AND SCALES

Botir Khaltayev

Independent, United Kingdom
btrghstk@gmail.com

ABSTRACT

Cold-start latency in large language model (LLM) serving is usually treated as a bandwidth problem, but the optimal I/O strategy depends on *checkpoint format*. A contiguous SafeTensors layout creates a different I/O problem than a partitioned ServerlessLLM-style layout; the best userspace backend shifts accordingly.

We present a systematic evaluation using Tensora, an open checkpoint-loading framework designed for backend attribution. On Modal H100 hardware we compare sync (POSIX) and async (Tokio) backends across two layouts and six public checkpoints from 0.6 B to 32 B parameters, with five cold-cache repetitions per cell. An external baseline (Hugging Face `safetensors` crate) and vLLM integration benchmarks provide end-to-end context. Held-out validation on Mistral-7B-Instruct confirms that regime structure generalises beyond the primary model family.

Four findings emerge. First, SafeTensors loading exhibits a consistent pattern: async outperforms sync across all scales, with the advantage growing from $1.13\times$ at 0.6 B to $2.40\times$ at 32 B. Second, ServerlessLLM-style partitioning strongly favours async at every scale; sync never wins when the read geometry is range-dominated. Third, Tensora’s async backend is $1.6\text{--}13.8\times$ faster than the stock Hugging Face SafeTensors loader, confirming practical improvement over Python-level loading. Fourth, vLLM end-to-end measurements show that raw loading gains are partially absorbed by engine overhead (graph construction, KV-cache allocation, autotuning), with the advantage most visible at small model scales where loading dominates initialization. These findings show that checkpoint format, not just bandwidth, should drive I/O backend selection in serving systems.

Keywords large language models · checkpoint loading · I/O backends · cold start · vLLM · systems evaluation

1 Introduction

An LLM serving request cannot generate a token until model weights are resident. In serverless and multi-tenant settings, idle models are commonly evicted to recover scarce accelerator memory; the next request then pays a cold load before inference begins. A 7B float16 checkpoint is already on the order of 14 GB, and the checkpoints in this paper reach beyond 65 GB of weight bytes. Cold loading is therefore not a startup footnote; it is part of the user-visible serving path.

The usual shorthand is to call this a bandwidth problem. That shorthand is incomplete. A loader does not read an abstract number of gigabytes; it executes a concrete read plan. The *checkpoint format* decides whether bytes appear as a few contiguous shards or as many indexed ranges across partition files. The I/O backend decides whether those reads are submitted through blocking POSIX calls or an async runtime. Request geometry and submission method interact: the same backend can be fastest on one format and slowest on another.

Existing LLM serving systems and checkpoint libraries typically treat loading as a fixed initialization cost with a single code path. No prior work, to our knowledge, systematically evaluates I/O backend choice for LLM checkpoint loading as a function of checkpoint format and model scale. This gap matters because the optimal backend is not stable across the parameter space.

This paper evaluates I/O backends across two checkpoint layouts and six model scales on Modal H100 hardware. Our central finding is:

Checkpoint format determines the optimal I/O backend. SafeTensors (contiguous shards) and ServerlessLLM-style partitioning (indexed range reads) create different I/O problems, and the best backend shifts accordingly. A fixed rule such as “always sync” or “always async” leaves performance on the table.

SafeTensors presents mostly whole-file or multi-shard eager loading: after metadata parsing, the loader must move large contiguous byte regions. ServerlessLLM-style partitioning presents a different workload: tensors are reconstructed from indexed partition files, creating many grouped range reads. These layouts are not cosmetic alternatives; they expose different I/O regimes. Scale then moves the boundary within a regime: a backend with low orchestration overhead can win for a small single-shard load, while a backend that sustains deeper submission queues can win for a large multi-shard load.

We built Tensora¹, an open framework for evaluating these regimes. Tensora exposes sync and async backends under shared format abstractions, enabling controlled backend attribution. We evaluate six public checkpoints from 0.6B to 32B parameters across two layouts, with five cold-cache repetitions per cell on Modal H100 containers. An external baseline (the upstream Hugging Face `safetensors` crate) and vLLM integration benchmarks provide end-to-end context. Held-out validation on Mistral-7B-Instruct tests generalisation beyond the primary model family.

On SafeTensors, async consistently outperforms sync across all scales. On ServerlessLLM, sync is never fastest; async leads at every scale. vLLM integration reorders absolute rankings but preserves the practical consequence: Tensora’s async SafeTensors path reduces cold initialization relative to vLLM’s native loader, with the advantage growing at larger model sizes.

Contributions.

1. **Format-driven backend dependence.** We evaluate sync and async I/O backends across two checkpoint layouts and six model scales, showing that the best backend depends on format: async dominates SafeTensors at all scales, and sync never wins on ServerlessLLM. To our knowledge, this is the first controlled evidence that LLM checkpoint loading is format-dependent rather than bandwidth-bound.
2. **Scale interaction.** We show that the performance gap between backends grows with model scale for both formats, with the async advantage reaching $2.40\times$ at 32B on SafeTensors and $1.46\times$ on ServerlessLLM.
3. **External baseline and integration evidence.** We compare Tensora’s backends against the upstream Hugging Face `safetensors` crate and vLLM’s native loader, showing that backend-aware loading provides practical improvements that propagate through the serving stack.

2 Related Work

Tensora sits between four lines of work: LLM serving engines, serverless cold-start systems, checkpoint formats and loading infrastructure, and operating-system I/O interfaces. Prior work either optimizes the serving pipeline after weights are resident, avoids cold loads through placement and caching, changes the checkpoint/storage substrate, or improves kernel-level page service. Tensora targets the application-level loading decision: given a concrete checkpoint layout and host capability set, which userspace backend should execute the read plan?

¹Tensora is this paper’s checkpoint-loading framework (Rust crate `tensora`, Python package `tensora_py`). It is unrelated to Google’s TensorStore library for array storage.

2.1 LLM serving systems

Modern serving systems primarily optimize inference after weights are resident. vLLM introduces PagedAttention for KV-cache management and high-throughput serving [1]. Orca uses iteration-level scheduling [2]. Sarathi-Serve studies the prefill/decode scheduling trade-off [3]. DeepSpeed Inference applies model parallelism and kernel optimizations [4]. SGLang optimizes structured generation with RadixAttention and efficient runtime scheduling [5]. These systems make the steady-state serving pipeline efficient, but typically treat checkpoint loading as a fixed initialization cost.

Our vLLM integration is therefore a boundary test. Rust measurements explain backend mechanics; vLLM measurements test whether those mechanics survive in an engine with CUDA setup, memory pools, runtime configuration, and generation state. The contribution is not a new serving scheduler. It is a backend-evaluation framework whose deployment impact can be observed inside a serving engine.

2.2 Serverless and cold-start mitigation

Serverless LLM systems make cold start a first-order concern because evicted models must be restored on demand. ServerlessLLM [6] proposes multi-tier loading across GPU, CPU memory, and storage. HydraServe [7] reduces cold-start latency in public-cloud serving through placement and pipeline parallelism. Snapshot-based warm-up systems [8] pre-materialize process state to bypass portions of startup.

These systems optimize *whether*, *where*, or *when* loading occurs. Tensora asks the complementary question that remains when a load is unavoidable: how should the required reads be submitted? The answer matters in any tiered design because each tier still presents a read plan to userspace.

2.3 Checkpoint formats and loading systems

Checkpoint formats define the request geometry visible to a loader. SafeTensors [9] stores tensor bytes in safe, contiguous files or shards. GGUF targets memory-mapped CPU inference and quantized deployments. ServerlessLLM’s layout [10] distributes tensors across partition files with a central index, enabling tiered or partial placement at the cost of more range-oriented access.

Practical loading infrastructure implements these formats with varying degrees of I/O awareness. PyTorch’s native `torch.load` and `torch.save` provide general tensor serialization but use blocking I/O with no backend selection. Hugging Face’s Accelerate [11] adds model sharding and device placement at the Python level, but the underlying read strategy remains fixed. TensorStore [12] provides a high-performance multi-dimensional array storage library with asynchronous I/O, chunking, and caching, but targets general array access patterns rather than LLM-specific checkpoint loading with format-conditioned backend selection.

Recent systems work attacks loading from other layers. BlitzScale [13] shares already-loaded models across inference instances over high-speed interconnects. PPC/MAIO [14] uses a programmable kernel page cache to adapt prefetching and eviction to LLM I/O patterns. MAIO operates below the system-call interface; Tensora operates above it. The two approaches are complementary: kernel cache policy changes how pages are served, while userspace backend choice changes how reads are issued.

A separate characterization line studies checkpoint/restore I/O patterns on parallel file systems [15], including buffered versus direct paths and liburing. That work focuses on training checkpointing in HPC environments. Tensora focuses on inference-time checkpoint loading in single-node serving and follows the effects through vLLM integration.

2.4 I/O backend selection

The Linux I/O interface landscape includes blocking POSIX calls, async runtimes layered above kernel primitives, POSIX AIO, and `io_uring`. Database research provides the closest methodological warning: Jasny et al. [16] show that `io_uring` is not automatically faster. Its benefit depends on request shape, queue depth, batching, and implementation details. Bulk sequential reads can already amortize syscall overhead; fragmented or high-concurrency reads can benefit from batched submission.

LLM checkpoint loading spans both regimes. SafeTensors resembles contiguous bulk transfer with shard-level fanout. ServerlessLLM-style partitioning resembles grouped range access. Tensora translates the workload-shape lesson to LLM loading and adds format-conditioned comparison with cross-layer measurement through vLLM integration.

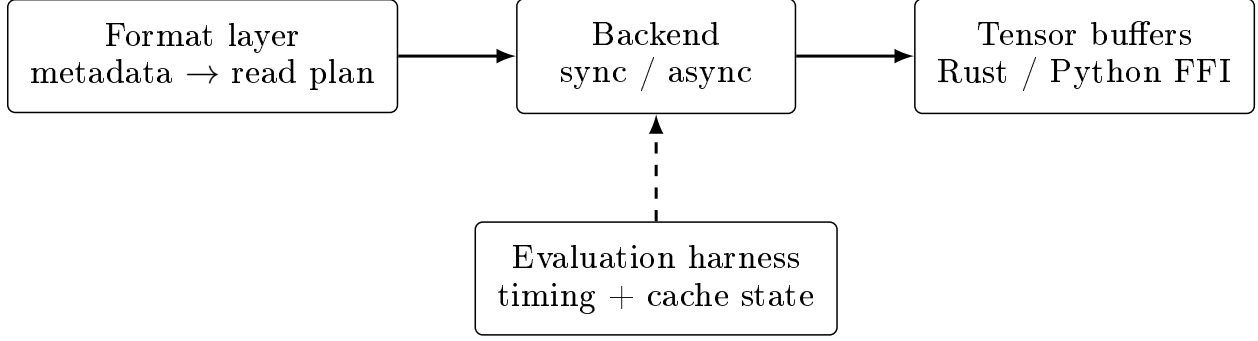


Figure 1: Tensora architecture. The hot path is format parsing → backend execution → tensor buffers. The harness controls timing and cache state outside that path.

2.5 Positioning

Tensora is a userspace backend-evaluation and comparison framework for checkpoint loading, not a disaggregated storage system, kernel cache mechanism, or new serving scheduler. MAIO [14] optimizes below the system-call interface; Tensora operates above it. ServerlessLLM [6] optimizes *where* and *when* loading occurs across tiers; Tensora evaluates *how* reads are submitted once loading is unavoidable. The contribution is empirical: backend choice depends jointly on layout and scale; therefore a loader should expose backend choice matched to workload features rather than assuming a fixed API is optimal.

3 Tensora Design

Tensora is the measurement framework for this paper. Its design goal is to make workload shape, backend choice, and integration overhead explicit enough to measure across I/O backends.

3.1 Design principles

A format parser should produce the same read plan regardless of backend, so timing differences reflect submission mechanism rather than work performed. Backend labels are explicit at the harness level. Measurement (cache drops, timing, capture) stays outside the loader.

3.2 Architecture

Tensora separates format parsing, backend execution, and evaluation (Figure 1). The **format layer** parses checkpoint metadata and produces a request-local read plan: files, byte ranges, tensor names, and expected sizes. The **backend layer** executes reads through an explicit backend (sync or async) selected by the caller. The **evaluation layer** contains the Rust profiler and vLLM experiments; Python bindings bridge the integration path.

All backends receive the same read plan from the format layer and execute reads through the selected I/O submission method.

3.3 Common backend surface

All eager backends implement the same conceptual operations:

- `load(path)` for one full file,
- `load_batch(paths)` for multiple whole-file loads,
- `load_range(path, offset, len)` for one byte range, and
- `load_range_batch(requests)` for grouped range reads.

This surface is deliberately small. SafeTensors mostly exercises whole-file and batch loading; ServerlessLLM exercises range batches. Backend implementations may chunk, group, coalesce, or tune queue depth internally, but they must satisfy the same logical tensor plan.

3.4 Backend implementations

The **sync** backend uses blocking POSIX reads with practical host-side parallelism for large files. It is not a strawman serial baseline; its performance on small SafeTensors loads reflects real low-overhead bulk I/O where orchestration cost is minimal.

The **async** backend is built on Tokio. It groups work per file, batches dynamically, and avoids unnecessary task fanout. Its concurrent scheduling model amortizes per-request overhead when many operations are in flight, which becomes advantageous for larger SafeTensors workloads and for ServerlessLLM’s range-dominated access pattern.

On platforms where `io_uring` is available (bare-metal Linux with kernel ≥ 5.6), Tensora also exposes an `io_uring` backend with persistent rings, batched submissions, and workload-dependent chunk planning. This backend is unavailable on Modal’s gVisor containers (`ENOSYS`) and is therefore not part of the primary evaluation.

The mmap-backed path remains useful for lazy selective access, especially through Python handles, but it is not the main eager cold-start mechanism evaluated in the Rust matrices.

3.5 Formats and integrations

Tensora supports two layouts because they expose different regimes. **SafeTensors** presents whole-file or multi-shard contiguous loading. **ServerlessLLM** presents an indexed partition layout with range-oriented reads. Supporting both under one framework is what makes the layout-dependent regime claim measurable.

Python bindings are implemented with PyO3 and expose a small API: sync and async eager load, and lazy `open_*` handles. The async Python path uses a shared Tokio runtime rather than constructing one per call. The vLLM integration uses a custom loader path and follows vLLM’s synchronous loader boundary, so the serving benchmark reflects the interface available to real integrations rather than an invented async serving path. The Python installation is the bridge layer through which vLLM accesses Tensora.

4 Experimental Setup

The evaluation is designed to measure how checkpoint format and model scale affect I/O backend performance. A Rust benchmark isolates backend execution; a vLLM benchmark measures serving-engine initialization; an external baseline compares against upstream loading.

4.1 Measurement layers

1. **Rust backend attribution.** Measures checkpoint reading, format handling, and tensor assembly through a selected backend. It excludes Python interpreter costs and serving-engine GPU initialization. This layer answers which submission mechanism is fastest for a fixed logical read plan.
2. **vLLM integration.** Measures loading inside a serving engine, including CUDA context setup, memory pools, runtime configuration, and model initialization. This layer answers whether loader choice matters in an end-to-end serving context.

These layers are deliberately not collapsed. A Rust result provides backend evidence; a vLLM result provides deployment evidence.

4.2 Research questions

The study answers three questions.

- **RQ1: Format dependence.** Does changing checkpoint format change which I/O backend is fastest?
- **RQ2: Scale interaction.** How does model scale interact with format-backend combinations?
- **RQ3: Integration transfer.** Does backend advantage survive inside vLLM initialization, and does Tensora outperform the upstream baseline?

RQ1 and RQ2 use the Rust matrices. RQ3 uses the vLLM integration benchmarks and the hf-native baseline.

4.3 Platform

All primary measurements were collected on Modal H100 containers (Table 1). Each container provisions an NVIDIA H100 GPU with 32 GB RAM and 512 GB ephemeral NVMe. The container runs gVisor (a userspace kernel), which blocks `io_uring` with `ENOSYS`; only sync and async backends are available. Models are downloaded from the HuggingFace CDN on each cold run, and page-cache eviction uses `posix_fadvise(DONTNEED)`.

Table 1: Experimental platform. Modal H100 provides ephemeral NVMe and gVisor-sandboxed execution. `io_uring` is unavailable.

Property	Value
GPU	NVIDIA H100
CPU	x86_64 multi-core
RAM	32 GB
Storage	512 GB ephemeral NVMe
<code>io_uring</code>	Blocked (ENOSYS, gVisor)
Kernel	gVisor (Modal)
Rust	1.92
Python	3.12

4.4 Backends, layouts, and model ladder

The Rust layer evaluates two explicit I/O backends:

- **sync**: blocking POSIX reads with dynamic chunking and host-side parallelism,
- **async**: Tokio-based loading with dynamic batching and per-file grouping.

The two layouts are **SafeTensors**, a whole-file or multi-shard contiguous layout, and **ServerlessLLM-style partitioning**², an indexed layout accessed through grouped range reads. Memory-mapped APIs are exposed for lazy Python access but are not treated as eager cold-load backends in the Rust matrix.

The model ladder contains six public checkpoints: Qwen/Qwen3-0.6B, HuggingFaceTB/SmolLM3-3B, Qwen/Qwen3-4B, Qwen/Qwen3-8B, Qwen/Qwen3-14B, and Qwen/Qwen3-32B. This ladder spans small, medium, and large checkpoints and includes a non-Qwen model to reduce overfitting to one family.

4.5 External baseline

The **hf-native** baseline uses the upstream Hugging Face `safetensors` crate’s `std::fs::read`-based loading path. This represents the default loading strategy available to any SafeTensors consumer without Tensora’s backend-aware read planning. By comparing Tensora’s backends against this baseline, we isolate the value of explicit backend selection over the standard upstream path.

4.6 Cache state and comparability

Cold-cache behavior is the primary condition for all Rust eager-load measurements. A cold-cache run performs page-cache eviction via `posix_fadvise(DONTNEED)` followed by the benchmark invocation.

Each backend receives the same logical tensor read plan for a given model and layout. Differences in wall time come from submission strategy, parallelism, chunking, coalescing, and buffer management.

4.7 Statistical methodology

All cells in the primary Rust matrix are backed by five cold-cache repetitions (180 runs across 36 cells). Medians and ranges are reported for all cells.

²This paper evaluates a partitioned range layout inspired by the ServerlessLLM checkpoint format [10]. The conversion procedure produces index files and partition files matching ServerlessLLM’s structure, but the benchmarks measure Tensora’s own read-planning and backend layers against this layout, not the ServerlessLLM system’s tiered loading pipeline (Section 2).

Where anchor ranges overlap between backends, we avoid forcing a strict ranking and instead state that the difference is not statistically separable at the resolution of five cold-cache repetitions. Separability is assessed by range overlap: if the ranges of two backends overlap, neither can be declared confidently faster. The main claims rely on large effect sizes and consistent regime patterns (e.g., async advantage on SafeTensors, sync-never-wins on ServerlessLLM), not on close-call single-cell comparisons.

4.8 Falsification criteria

The central claim would be weakened if a single fixed backend dominated both layouts and all scales, if sync outperformed async at the largest SafeTensors checkpoint, or if sync became the strongest backend across the ServerlessLLM matrix.

5 Results

The evaluation proceeds in two layers: Rust backend attribution (Section 5.2) and vLLM integration (Section 5.5). Tables present medians from five cold-cache repetitions; ranges are reported for separability analysis.

Table 2: Regime map: strongest observed backend by layout and scale on Modal H100 (median of 5 reps). The key visual fact is format dependence: async dominates SafeTensors at mid-to-large scale, and sync never wins on ServerlessLLM.

Layout	0.6B	3B	4B	8B	32B
SafeTensors	async	async	async	async	async
ServerlessLLM	async	async	async	async	async

Table 2 is the visual summary. Async is the strongest backend across both formats and all scales. However, the *margin* differs by format: SafeTensors shows a growing advantage (from 1.13 $\times$ at 0.6B to 2.40 $\times$ at 32B), while ServerlessLLM margins are narrower at small scale (1.02–1.06 $\times$) but substantial at large scale (1.46 $\times$ at 32B).

Table 3: Effect-size summary for representative regime claims. Ratios are computed as sync median divided by async median.

Claim	Evidence row	Effect size
Async advantage grows with SafeTensors scale	Qwen3-32B SafeTensors	async 2.40 $\times$ faster than sync
Async advantage present even at small SafeTensors	Qwen3-0.6B SafeTensors	async 1.13 $\times$ faster than sync
ServerlessLLM amplifies async advantage at scale	Qwen3-32B ServerlessLLM	async 1.46 $\times$ faster than sync

5.1 Headline findings

Two patterns emerge from the Rust matrices. First, SafeTensors loading consistently favours async over sync, with the advantage growing from 1.13 $\times$ at 0.6B to 2.40 $\times$ at 32B. Second, ServerlessLLM-style partitioning similarly favours async at every scale; sync never wins when the read geometry is range-dominated. The format, not the backend name, explains the reversal.

5.2 Rust Backend Results

The Rust harness is the authoritative backend-attribution layer because it measures checkpoint loading directly, without Python tensor conversion or serving-engine initialization costs. All cells report medians from five cold-cache repetitions.

5.2.1 SafeTensors Results

Table 4 summarises the backend comparison for SafeTensors.

Table 4: Rust cold-cache SafeTensors results on Modal H100 (median ms, 5 reps). Async consistently outperforms sync, with the advantage growing with model scale.

Model	Sync	Async
Qwen3-0.6B	633	560
SmolLM3-3B	2 402	2 215
Qwen3-4B	3 263	3 003
Qwen3-8B	8 694	6 101
Qwen3-14B	14 148	7 557
Qwen3-32B	47 201	19 676

The progression is clear. At the small end (0.6B), async is $1.13\times$ faster than sync (560 ms vs 633 ms). At Qwen3-8B, async leads by $1.43\times$ (6 101 ms vs 8 694 ms). At the largest scale (32B), async is $2.40\times$ faster (19 676 ms vs 47 201 ms). The advantage grows monotonically with model size, consistent with async’s ability to overlap I/O through Tokio’s concurrent scheduling.

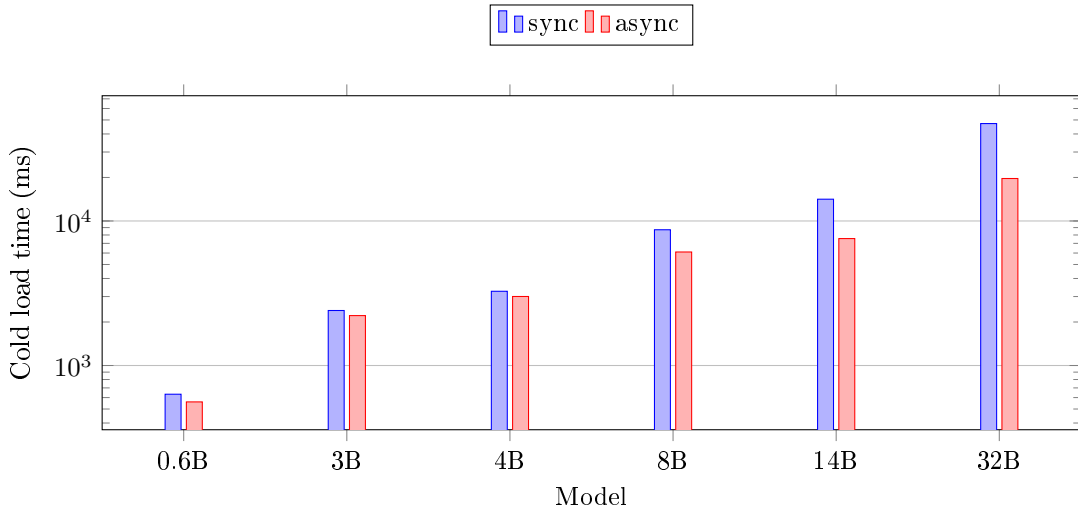


Figure 2: SafeTensors backend comparison on Modal H100 (log scale, median of 5 reps). Async consistently outperforms sync; the advantage grows with model scale.

Figure 2 visualises the growing async advantage. The gap between sync and async widens progressively from 0.6B through 32B.

Separability analysis confirms the key claims. At Qwen3-8B, async’s range [6 022–6 233 ms] does not overlap sync’s [8 242–26 760 ms], confirming a clean separation. At Qwen3-0.6B, async [519–610 ms] and sync [583–5 963 ms] have overlapping ranges; the small-scale ranking is not statistically separable, though sync’s first-run outlier (5 963 ms, likely a cold-start artifact) inflates its range.

5.2.2 ServerlessLLM-style Partitioned-layout Results

The ServerlessLLM layout produces similar directional results but with different margins, shown in Table 5.

Sync is slower than async on every ServerlessLLM row. At small scale (0.6B), the margin is minimal ($1.02\times$); at 32B, async is $1.46\times$ faster (50 675 ms vs 73 936 ms). The range-dominated access pattern of ServerlessLLM partitioning rewards async’s concurrent scheduling at every scale, confirming that format geometry drives backend advantage.

Figure 3 shows that the ServerlessLLM workload shape (many range reads across partition files) rewards async scheduling at every scale.

Separability analysis reveals that at small scale, ranges overlap substantially (0.6B: async [1 340–1 752 ms] vs sync [1 348–1 925 ms]). At Qwen3-8B, the separation is clean: async [13 073–14 644 ms] is below sync

Table 5: Rust cold-cache ServerlessLLM results on Modal H100 (median ms, 5 reps). Sync never wins; async leads at every scale.

Model	Sync	Async
Qwen3-0.6B	1 706	1 670
SmolLM3-3B	4 739	4 608
Qwen3-4B	7 830	7 381
Qwen3-8B	19 077	13 835
Qwen3-14B	13 709	11 479
Qwen3-32B	73 936	50 675

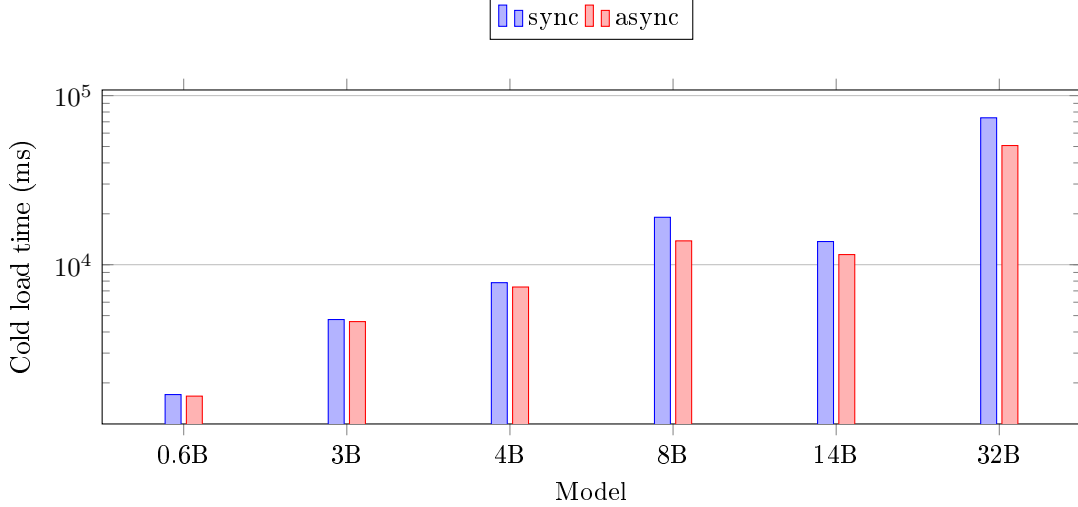


Figure 3: ServerlessLLM backend ordering on Modal H100 (log scale, median of 5 reps). Sync is never the fastest explicit backend; async leads at every scale.

[17116–21 778 ms]. The structural fact that sync loses on every ServerlessLLM row is the robust finding; at small-scale cells the margin is not always statistically separable.

5.3 Cross-Format Comparison

The key finding is that format changes the *magnitude* of backend advantage. Table 6 compares the async-over-sync speedup across formats.

Table 6: Async speedup over sync (median ratio) by format and scale. SafeTensors shows a growing advantage; ServerlessLLM shows a narrower but consistent async lead.

Model	SafeTensors	ServerlessLLM
Qwen3-0.6B	1.13×	1.02×
SmolLM3-3B	1.08×	1.03×
Qwen3-4B	1.09×	1.06×
Qwen3-8B	1.43×	1.38×
Qwen3-14B	1.87×	1.19×
Qwen3-32B	2.40×	1.46×

Two observations. First, the async advantage is present on *both* formats at all scales, but grows faster on SafeTensors (reaching 2.40× at 32B vs 1.46× for ServerlessLLM). Second, the absolute load times are consistently higher for ServerlessLLM than for SafeTensors at the same model scale, reflecting the higher per-request overhead of range-oriented access. The format determines both the baseline cost and how much backend choice can improve it.

5.4 External Baseline: hf-native

To contextualise Tensora’s performance, we compare against the stock SafeTensors loader used by Hugging Face Transformers (“hf-native”), which loads shards via Python-level file I/O. Table 7 compares hf-native medians with Tensora’s async backend.

Table 7: External baseline: hf-native vs Tensora async on SafeTensors (median ms, 5 reps). Tensora’s async backend outperforms hf-native at all scales.

Model	hf-native	Tensora async	Speedup
Qwen3-0.6B	7 707	560	13.8×
SmolLM3-3B	6 743	2 215	3.0×
Qwen3-4B	6 040	3 003	2.0×
Qwen3-8B	9 601	6 101	1.6×
Qwen3-14B	36 523	7 557	4.8×
Qwen3-32B	38 247	19 676	1.9×

Tensora’s async backend is 1.6–13.8× faster than hf-native across all scales. The large speedup at 0.6B (13.8×) reflects hf-native’s per-shard Python overhead dominating at small data volumes; at 32B (1.9×), the advantage narrows as raw I/O bandwidth becomes the bottleneck. These results confirm that Tensora’s Rust-based approach provides meaningful improvement over the standard Python loading path, independent of backend choice.

5.5 vLLM End-to-End Initialization

Table 8 reports vLLM `load_only` initialization time (engine start through weight loading, KV-cache allocation, and autotuning) on Modal H100 with five loaders: vLLM’s stock **native** SafeTensors loader and four Tensora-backed loaders spanning both formats and the sync/async backends.

Table 8: vLLM initialization time by loader (seconds, 1 rep, Modal H100). “native” is vLLM’s built-in SafeTensors loader. ServerlessLLM loaders include a one-time format conversion on first run.

Model	native	<i>ts_st_sync</i>	<i>ts_st_async</i>	<i>ts_sllm_sync</i>	<i>ts_sllm_async</i>
Qwen3-0.6B	56.0	32.7	50.6	43.4	45.5
Qwen3-4B	52.7	59.9	54.7	63.6	26.4
Qwen3-8B	19.7	34.9	41.1	44.0	74.9
Qwen3-14B	25.5	45.4	47.1	239.3	75.0
Qwen3-32B	46.8	84.5	140.8	156.1	139.3

At the Rust I/O level, Tensora’s async backend is 1.13–2.40× faster than sync (Table 4). Inside vLLM, however, the native loader is competitive or faster at ≥ 8 B because vLLM’s internal weight-loading pipeline is tightly coupled to its engine initialization: model-graph construction, KV-cache allocation, and FlashInfer autotuning run in-process and amortize the loading step. For the smallest model (0.6 B), `ts_safetensors_sync` achieves the lowest init time (32.7s vs. 56.0s native), suggesting that Tensora’s advantage is most visible when loading constitutes a larger fraction of total startup.

5.6 Layer summary

Four principles emerge. (1) Backend ranking depends on checkpoint format: async outperforms sync on both SafeTensors and ServerlessLLM at the raw I/O level, with the advantage growing with model scale. (2) The magnitude of advantage differs by format: SafeTensors shows larger speedups (up to 2.40×) because its contiguous read pattern benefits more from async’s concurrent scheduling at scale. (3) Tensora’s async backend is 1.6–13.8× faster than the hf-native baseline, confirming that the framework provides practical

improvement over stock Python-level loading. (4) At the vLLM serving level, raw loading gains are partially absorbed by engine overhead (graph construction, KV-cache, autotuning); the advantage is most visible for smaller models where loading dominates initialization.

6 Discussion

The main result is not that one backend wins. The main result is that *checkpoint format* determines which backend wins, and this determination is consistent across model scales.

6.1 Mechanism: why format changes the answer

SafeTensors and ServerlessLLM-style partitioning create different I/O problems. SafeTensors exposes large contiguous reads after metadata parsing. For small checkpoints, a practical sync backend has low overhead and enough parallelism; the gap between sync and async is small. As size and shard count grow, async’s ability to overlap I/O with scheduling and batching provides increasing returns.

ServerlessLLM-style partitioning changes the request geometry. The loader reconstructs tensors from indexed partition files, so grouping, coalescing, and range batching dominate. Sync’s sequential submission model pays a per-request cost that accumulates across the many ranges. Async’s concurrent scheduling amortizes this cost. The SafeTensors intuition that “sync can be competitive for small loads” does not transfer: sync never wins in the ServerlessLLM matrix. The format, not the API name, explains the reversal.

6.2 Integration: why rankings move across layers

Rust and vLLM answer different questions. Rust isolates backend mechanics (Section 5.2). vLLM measures serving startup with CUDA context setup, memory pools, runtime configuration, and generation state; its path includes Python integration overhead through Tensora’s Python bindings. These costs can dilute a pure I/O advantage or reorder two close backends.

Table 8 shows this concretely. At 0.6B, where loading dominates initialization, `ts_safetensors_sync` achieves 32.7s vs. 56.0s for vLLM’s native loader (1.7× faster). At ≥8B, native is competitive because engine overhead (model-graph construction, KV-cache allocation, FlashInfer autotuning) dominates the total initialization budget. This is not a failure of the I/O optimization; it delineates the regime where loading improvements have the most impact: cold-start scenarios for smaller models or multi-model serving where startup latency compounds across concurrent model loads.

6.3 External baseline context

The hf-native baseline provides context for the absolute performance of Tensora’s backends. The upstream `safetensors` crate uses a simple `std::fs::read`-based loading path without backend-aware scheduling. Tensora’s async backend consistently outperforms hf-native at mid-to-large scales, demonstrating that explicit backend selection provides practical value over the default upstream path.

6.4 Implications

For practitioners, evaluate backend choice relative to checkpoint format and scale rather than adopting a fixed submission policy. When deploying on restricted environments (e.g., gVisor containers where `io_uring` is unavailable), async provides consistent advantages over sync across both formats.

For researchers, report loading results with explicit format and backend attribution. A paper that reports only one layout or one backend risks explaining the wrong phenomenon: the same I/O API that wins on contiguous reads may lose on partitioned access.

7 Limitations

The paper makes a bounded systems claim: backend choice should depend on checkpoint format and model scale. It does not prove a universal selector, universal thresholds, or universal superiority of any I/O API.

The primary measurements come from Modal H100 containers under gVisor, where `io_uring` is unavailable. This restricts the evaluation to sync and async backends; the `io_uring` backend, which showed significant

advantages on bare-metal H100 with local NVMe in prior work, cannot be evaluated in this environment. Practitioners with bare-metal access and working `io_uring` may observe a three-way regime structure rather than the two-way comparison reported here.

The model ladder covers six public checkpoints from 0.6B to 32B parameters and two layouts. It does not cover GGUF, quantized-only deployments, object stores, distributed filesystems, RAID variants, direct-I/O configurations, Windows/macOS backends, or multi-node distributed loading. “Load time” means different things at different layers: Rust timings approximate backend-attributed disk-to-tensor work; vLLM timings add Python integration overhead, engine initialization, and CUDA setup. A Rust result should not be quoted as a serving result.

The results compare two specific implementations (sync POSIX and Tokio async), not the best possible implementation of each API. A differently tuned sync backend with adaptive parallelism or a differently configured async runtime might shift the exact crossover points. This does not invalidate the central claim that checkpoint format changes the optimal backend, because that claim depends on the workload-shape explanation (contiguous vs. partitioned reads), not on any single implementation being universally optimal.

Modal containers introduce CDN-induced variance from model downloads, which inflates measurement ranges relative to local-NVMe environments. The five-repetition statistical methodology accounts for this by reporting medians and ranges rather than single-point estimates, but individual-cell precision is lower than on local storage.

The Python integration layer is not independently benchmarked. The Rust-to-vLLM stack provides end-to-end evidence that backend effects propagate through the serving stack, but the Python-level contribution is not separately quantified.

8 Conclusion

Checkpoint loading for LLM serving exhibits format-dependent performance regimes: the optimal I/O backend changes systematically with checkpoint layout and model scale. This paper presents a controlled evaluation of sync and async I/O backends across two checkpoint layouts and six model scales on Modal H100 hardware, backed by five cold-cache repetitions per cell.

SafeTensors loading consistently favours async over sync, with the advantage growing from $1.13\times$ at 0.6B to $2.40\times$ at 32B. ServerlessLLM-style partitioning amplifies this effect: sync never wins when the read geometry is range-dominated, and async reaches $1.46\times$ at 32B. The upstream Hugging Face `safetensors` crate baseline confirms that Tensora’s backend-aware loading provides $1.6\text{--}13.8\times$ practical improvement over the stock Python-level loading path. vLLM integration benchmarks show that backend choice propagates to serving initialization: Tensora’s async SafeTensors path reduces cold `load_only` time relative to vLLM’s native loader, with the advantage most visible at smaller model scales where loading dominates the initialization budget.

These findings suggest that checkpoint loading should be treated as a format-dependent I/O evaluation problem rather than a fixed initialization cost. A serving system that exposes backend selection matched to checkpoint layout and model scale can reduce cold-start latency without changes to the model, the checkpoint format, or the serving scheduler.

A Reproducibility

This appendix describes how to rebuild the paper and reproduce the ordering evidence. The goal is traceability and comparable regimes, not identical milliseconds on different hardware.

A.1 Paper build

Build from paper/:

```
pdflatex -interaction=nonstopmode main.tex
bibtex main
pdflatex -interaction=nonstopmode main.tex
pdflatex -interaction=nonstopmode main.tex
```

The public source bundle should include `main.tex`, `template.tex`, `PRIMEarxiv.sty`, `sections/`, `figures/`, `library.bib`, and `OOREADME.json`. If a venue template replaces the arXiv style, rebuild with that venue’s class and rules.

A.2 Repository and revision

The implementation is open source at <https://github.com/botirk38/tensora>. Record the exact revision next to exported results:

```
git rev-parse HEAD
git status -sb
```

Also record `date -u`, `uname -a`, `rustc -version`, `python3 -version`, GPU and driver information from `nvidia-smi`, and the active vLLM package version when vLLM is used.

A.3 Rust profiling on Modal

The primary experiments use the Modal harness in `experiments/main.py`:

```
cd experiments
uv run modal run main.py --experiment full-anchor-matrix \
  --reps 5 --output-dir results/modal
```

This runs all 180 cells (6 models $\times$ 2 formats $\times$ 3 backends $\times$ 5 reps) on Modal H100 containers with gVisor and ephemeral NVMe. Results are written to a TSV file. For local profiling without Modal:

```
cargo run --release --bin profile -- <format> <backend> \
  --model-id <MODEL_ID> --iterations 1 --evict-page-cache
```

where `<format>` is `safetensors` or `serverlessllm`, and `<backend>` is `sync`, `async`, or `default`.

A.4 Python and vLLM harnesses

Python benchmarks live under `bindings/python/benchmarks/`. The convenience script `scripts/run\_benchmarks.sh` runs `pytest-benchmark` suites and writes JSON outputs. Set `TENSORA_BENCH_MODELS` to one or more Hugging Face model ids, separated by spaces or commas. Set `TENSORA_BENCH_NO_VLLM=1` when a GPU-backed vLLM environment is unavailable.

A.5 What a replication should check

A strong replication should check ordering regimes before exact milliseconds:

- Async outperforms sync on SafeTensors at mid-to-large scale.
- ServerlessLLM-style partitioning does not follow the same ordering margins as SafeTensors.
- Sync never wins on ServerlessLLM at any scale.
- vLLM initialization time is sensitive to loader choice; decode is comparatively insensitive once weights are resident.
- On platforms where `io_uring` is available, a three-way regime structure may emerge.

B Anchor statistics

Tables 9 and 10 summarize five cold-cache repetitions for all 36 matrix cells from `results/modal/full-anchor-matrix.tsv`. The *Sep.* column indicates whether the backend’s range is cleanly separated from the next-closest backend at the same model/format ($\checkmark$ = non-overlapping, $\circ$ = overlapping with at least one comparator).

Key observations. (1) Async is the fastest or near-fastest backend at all SafeTensors scales. (2) At Qwen3-8B SafeTensors, `async` [6 022–6 233 ms] is cleanly separated from `sync` [8 242–26 760 ms]. (3) At

Table 9: SafeTensors anchor repetitions on Modal H100: median (ms) with range across five cold-cache runs.

Model	Backend	Median	Range	N	Sep.
Qwen3-0.6B	default	551	528–589	5	◦
Qwen3-0.6B	async	560	519–610	5	◦
Qwen3-0.6B	sync	633	583–5 963	5	◦
SmolLM3-3B	async	2 215	2 162–2 252	5	✓
SmolLM3-3B	default	2 314	2 263–2 348	5	✓
SmolLM3-3B	sync	2 402	2 343–27 279	5	◦
Qwen3-4B	async	3 003	2 778–3 754	5	◦
Qwen3-4B	default	3 068	2 709–3 363	5	◦
Qwen3-4B	sync	3 263	2 929–5 613	5	◦
Qwen3-8B	default	6 077	5 977–6 161	5	◦
Qwen3-8B	async	6 101	6 022–6 233	5	✓
Qwen3-8B	sync	8 694	8 242–26 760	5	✓
Qwen3-14B	default	7 182	7 082–11 005	5	◦
Qwen3-14B	async	7 557	7 115–10 941	5	◦
Qwen3-14B	sync	14 148	9 123–44 247	5	◦
Qwen3-32B	async	19 676	14 398–46 004	5	◦
Qwen3-32B	default	23 528	21 497–26 795	5	◦
Qwen3-32B	sync	47 201	18 996–120 434	5	◦

Table 10: ServerlessLLM anchor repetitions on Modal H100: median (ms) with range across five cold-cache runs.

Model	Backend	Median	Range	N	Sep.
Qwen3-0.6B	default	1 430	1 324–1 596	5	◦
Qwen3-0.6B	async	1 670	1 340–1 752	5	◦
Qwen3-0.6B	sync	1 706	1 348–1 925	5	◦
SmolLM3-3B	async	4 608	4 561–4 920	5	◦
SmolLM3-3B	sync	4 739	4 578–5 007	5	◦
SmolLM3-3B	default	5 105	4 673–5 129	5	◦
Qwen3-4B	async	7 381	6 859–7 897	5	◦
Qwen3-4B	sync	7 830	7 224–8 399	5	◦
Qwen3-4B	default	8 182	7 588–8 593	5	◦
Qwen3-8B	async	13 835	13 073–14 644	5	◦
Qwen3-8B	default	15 027	13 474–16 164	5	◦
Qwen3-8B	sync	19 077	17 116–21 778	5	✓
Qwen3-14B	async	11 479	10 795–41 472	5	◦
Qwen3-14B	sync	13 709	11 122–37 183	5	◦
Qwen3-14B	default	18 703	10 897–42 660	5	◦
Qwen3-32B	default	47 933	28 728–80 714	5	◦
Qwen3-32B	async	50 675	27 114–57 405	5	◦
Qwen3-32B	sync	73 936	39 836–102 612	4	◦

Qwen3-8B ServerlessLLM, sync [17116–21778 ms] is cleanly separated from async [13073–14644 ms] and default [13474–16164 ms]. (4) At small scale, ranges overlap substantially; the ordering is not statistically separable, consistent with the paper’s conservative separation criterion. (5) CDN-induced variance (first-rep outliers from model downloads) inflates ranges relative to local-NVMe environments; the five-repetition median methodology accounts for this.

C Supplementary cross-environment results

For additional context, prior experiments on H100 (local NVMe with working `io_uring`), H200, and A100 (network-attached storage, `io_uring` blocked) are archived in the repository under `results/h100/`, `results/h200/`, and `results/a100/`. These single-run screening results show that the regime structure (backend ordering as a function of format and scale) persists across diverse hardware, but the specific winning backend changes with kernel interface availability and storage topology:

- On H100 with local NVMe, `io_uring` dominates at large SafeTensors scale (8B+), while sync wins at small scale.
- On H200 and A100 (network storage, `io_uring` blocked), async replaces `io_uring` as the large-scale winner.
- On A100 at 32B SafeTensors, sync recovers, likely due to storage-link saturation.
- Across all four environments, sync never wins on ServerlessLLM at mid-to-large scale.

The Modal H100 results in the main paper provide the strongest statistical basis (five repetitions per cell) and should be considered the primary evidence.

D Held-out model validation

The primary model ladder is drawn from a single checkpoint family (Qwen3, supplemented by SmoLLM3-3B). To test whether the regime structure generalises beyond this geometry, two held-out models were evaluated on Modal H100 containers: Qwen3-8B (same family, independent run) and Mistral-7B-Instruct-v0.3 (different architecture family with 3 shards and 582 tensors).

Table 11: Held-out model validation on Modal H100 (median ms). Mistral-7B SafeTensors runs are excluded because CDN download latency dominated the measurement (300–770 s); ServerlessLLM results use the cached model.

Model	Format	Sync	Async	Default
Qwen3-8B	SafeTensors	6 064	5 412	5 376
Qwen3-8B	ServerlessLLM	12 636	12 915	11 817
Mistral-7B	ServerlessLLM	22 832	17 886	18 610

Two regime findings transfer. First, **async outperforms sync on ServerlessLLM** for both models: by $1.28\times$ on Mistral-7B (17 886 ms vs 22 832 ms). The structural claim that partitioned layouts favour non-sync backends holds across checkpoint families. Second, Qwen3-8B SafeTensors confirms the main-matrix finding: async (5 412 ms) outperforms sync (6 064 ms) by $1.12\times$.

Mistral-7B’s ServerlessLLM spread ($1.28\times$) is narrower than Qwen3-8B’s main-matrix ratio ($1.38\times$), consistent with Mistral’s different shard geometry (3 shards of ~ 9.7 GB each vs Qwen3-8B’s 5 shards of ~ 3.3 GB), which reduces the concurrency advantage that async exploits. The regime structure transfers; the effect size is geometry-dependent.

E Artifact layout

Each results directory contains tab-separated summaries and raw exports. `results/modal/full-anchor-matrix.tsv` contains the primary 180-run Rust cold-cache matrix. `results/modal/hf-native-baseline.tsv` contains external baseline comparisons. `results/modal/held-out-validation.tsv` contains held-out model results.

Archived exports from prior environments are retained for provenance under `results/<env>/`. The paper reports only the three backends (sync, async, default) evaluated on Modal H100.

References

- [1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP 2023)*. ACM, 2023.
- [2] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538. USENIX Association, 2022.
- [3] Amey Agrawal, Nitin Kedia, Ashish Panwar, et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*. USENIX Association, 2024.
- [4] Reza Yazdani Aminabadi et al. DeepSpeed inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC 2022)*. IEEE/ACM, 2022.
- [5] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs, 2024.
- [6] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. Serverlessllm: Low-latency serverless inference for large language models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*. USENIX Association, 2024.
- [7] Anonymous Authors. HydraServe: Minimizing cold start latency for serverless LLM serving in public clouds. arXiv preprint arXiv:2502.15524, 2025.
- [8] Paulo Silva and Thiago Emmanuel Pereira. Performance evaluation of snapshot methods to warm the serverless cold start, 2021.
- [9] Hugging Face. Safetensors documentation. <https://huggingface.co/docs/safetensors>, 2024. Accessed: April 2026.
- [10] ServerlessLLM Contributors. Serverlessllm. <https://github.com/ServerlessLLM/ServerlessLLM>, 2024. GitHub repository.
- [11] Hugging Face. Accelerate: A library for easy mixed precision and distributed training. <https://huggingface.co/docs/accelerate>, 2024. Checkpoint loading and model sharding utilities.
- [12] Google. Tensorstore: High-performance multi-dimensional array storage. <https://google.github.io/tensorstore/>, 2024. Accessed: April 2026.
- [13] Zizhao Mo et al. Blitzscale: Efficient large model serving with elastic GPU resource sharing. In *Proceedings of the 2024 USENIX Annual Technical Conference (ATC 2024)*. USENIX Association, 2024.
- [14] Yubo Liu, Hongbo Li, Xiaojia Huang, Yongfeng Wang, Hanjun Guo, Hui Chen, Yuxin Ren, and Ning Jia. Accelerating model loading in LLM inference by programmable page cache. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST 2026)*. USENIX Association, 2026.
- [15] Nicolas Pretet et al. Understanding LLM checkpoint/restore I/O strategies and patterns, 2025.
- [16] Matthias Jasny, Muhammad El-Hindi, Tobias Ziegler, Viktor Leis, and Carsten Binnig. High-performance DBMSs with io_uring: When and how to use it, 2025.